Split Inference on a Heterogeneous ESP32 Fleet

Project Work in Internet of Things

Giacomo Antonelli

September 2026

Abstract

Split inference runs the first k layers of a neural network on a microcontroller and sends the intermediate activation tensor to a server that finishes the job. We built the full pipeline on three ESP32-class boards of differing capability and measured what the choice of k costs. Across 3,813 requests with zero corrupted deliveries, full offload beat every split point on every board, by 5.1× on an ESP32-S3 and 15.5× on an ESP32-CAM, and the weakest board could not host a split point at all. Cutting deeper made things worse rather than better: the deeper cut shrank the tensor from 4,668 B to 888 B but bought 2.7 ms of transfer with 26–164 ms of extra on-device inference. The per-stage breakdown shows why. Device compute dominates by two orders of magnitude, while transfer, the quantity split inference exists to reduce, costs 7–12 ms regardless of which cut produced it. A sensitivity analysis over the measured stages puts break-even at a server 51× slower than ours.

1 The question

For a network layer k, a split deployment pays three costs: on-device inference of the first k layers, transfer of the resulting activation tensor, and server inference of the remainder. Cutting later means more device compute and usually less data on the air. The premise of split inference is that some intermediate k beats both endpoints, and that the best k depends on the device.

This report tests that premise by measurement. It does not learn or adapt the split point: every policy here is a fixed rule, and the exhaustive sweep is the ground truth an adaptive orchestrator would have to beat.

2 System

2.1 Hardware

Node	Tier	Board	Memory
11	A	ESP32-S3-WROOM-1 N16R8	8 MB octal PSRAM @ 80 MHz, vector ISA
21	B	ESP32-CAM (ESP32-D0WD)	8 MB quad PSRAM @ 80 MHz, no vector ISA
31	C	ESP32-D0WD-V3, plain	no PSRAM, ∼150 KB usable SRAM
		x86 laptop (Core 7 150U)	soft-AP, orchestrator, server tail

Table 1: The fleet. All three MCUs run at 240 MHz.

All boards run ESP-IDF v5.5.3 with TensorFlow Lite Micro. The server is the same machine that builds the firmware: it runs the Wi-Fi access point (2.4 GHz, since ESP32 radios have no 5 GHz band), the orchestrator, and the tail inference. Its CPU is capped to 400 MHz and the process pinned to four cores. Section 6 states what that costs the conclusions.

2.2 Model and split points

MobileNetV2, width multiplier 0.35, $96 \times 96 \times 3$ input, full-int8 post-training quantization, two classes. Heads and tails are sliced from one set of weights at the flatbuffer level, with the boundary tensor's quantization parameters copied verbatim from head output to tail input. A test verifies bit-exact equality of int8 logits between full(x) and tail$_k$(head$_k$(x)) over 200 images at every cut, so accuracy is identical by construction and latency is the only thing that varies between policies.

Three split points: `k0` (no split, meaning JPEG to the server and the whole model on the server), `k_shallow` (after inverted-residual block 2), and `k_deep` (after block 6).

2.3 Transport and measurement

UDP with application-level fragmentation, CRC32 per tensor, NACK-based selective repeat over three rounds, and an abort path. One asyncio process on the server dispatches at most one request per node at a time. Every request writes one SQLite row decomposing end-to-end latency into five non-overlapping stages that sum exactly to the total: `device` (flash read plus head inference), `uplink`, `queue`, `server`, `residual`. Every number in this report is a query over those rows.

3 Method

Two kinds of run. The *sweep* measures every feasible *(node, cut)* pair, 200 requests each, one node at a time. That serialisation is what makes the resulting numbers properties of the cut rather than of what the other nodes happened to be doing. The *policy runs* then exercise four fixed rules with the whole fleet active: `k0`, `k_shallow`, `k_deep`, and `best`, which replays the sweep's per-node argmin.

Every board reads the same 50 images from flash, in the same order, so all policies see identical inputs.

Seven gates guard the procedure; each invalidates everything after it if it fails. They cover split correctness, ARQ integrity under injected loss, whole-harness verification against simulated nodes, per-tier arena feasibility, fleet stability, sample counts, and dispatch correctness.

4 Results

4.1 Which split points each board can host

Tier	Arena source	`k_shallow`	`k_deep`	Feasible cuts
A	8 MB octal PSRAM	149,972 B	158,120 B	`k0`, `k_shallow`, `k_deep`
B	8 MB quad PSRAM	142,296 B	150,396 B	`k0`, `k_shallow`, `k_deep`
C	100 KB internal	needs 138,240 B, has 98,480 B		`k0` only

Table 2: Arena required per head, measured on device at boot. Tier C is short by 39 KB and hosts no head at all: its firmware links none.

Two results here. First, the weakest board cannot split, so on that hardware the only available policy is full offload. Second, the same cut costs different arena on different silicon: 149,972 B on the S3 against 142,296 B on the ESP32 for identical weights and tensor shapes, because the S3 build uses vector kernels that allocate their own scratch.

Cut	Activation bytes	Mean bytes on air	Fragments	Retransmits/req
`k0`	1,775 (JPEG)	1,811	1.98	0.000
`k_shallow`	4,608	4,668	4.00	0.000
`k_deep`	864	888	1.00	0.000

Table 3: The data-inflation effect. Cutting shallow puts 2.6× more on the air than simply sending the JPEG; only the deep cut falls below it.

Node	Cut	n	Mean (ms)	p50 (ms)	p95 (ms)
11 (A)	`k0`	200	**32.6**	29.8	55.5
11 (A)	`k_shallow`	200	166.8	167.5	184.2
11 (A)	`k_deep`	200	190.0	189.9	209.6
21 (B)	`k0`	200	**36.0**	33.0	63.8
21 (B)	`k_shallow`	200	557.8	557.7	581.4
21 (B)	`k_deep`	200	719.9	720.3	743.6
31 (C)	`k0`	200	**41.5**	40.4	68.4

Table 4: The sweep: every feasible pair, measured in isolation. Zero failures.

4.2 Bytes on the air

4.3 Latency per node and split point

Full offload wins on every node, at the mean and at p95, by 5.1× on tier A and 15.5× on tier B. And `k_deep` is worse than `k_shallow` on both capable tiers, despite putting a fifth as much data on the air.

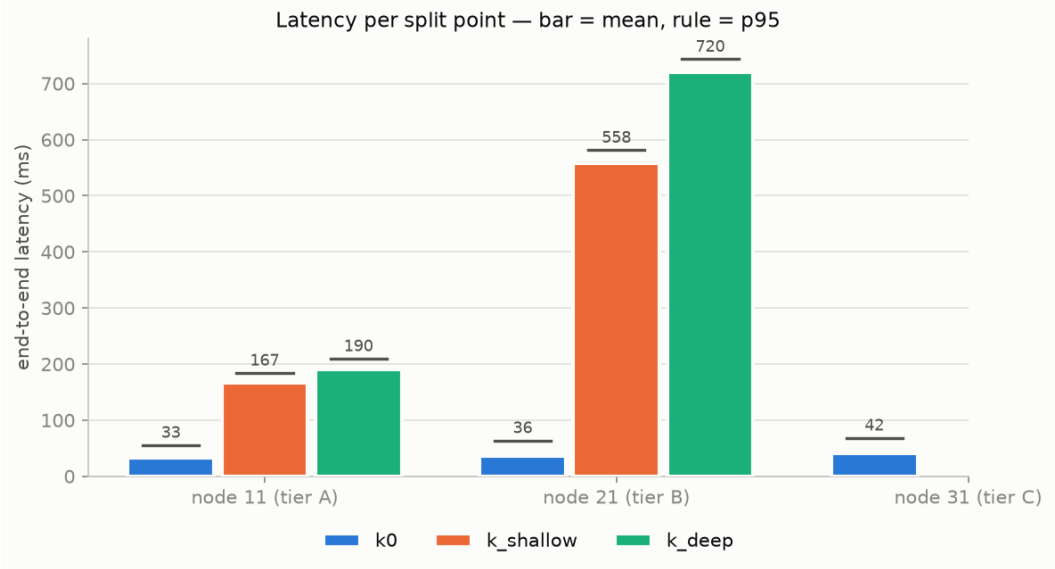

Figure 1: Latency per split point; bar is the mean, rule is p95.

4.4 Where the time goes

The `device` stage dominates every split cut and nothing else is within two orders of magnitude. Meanwhile `uplink` is almost constant across cuts, 7.0 to 11.9 ms, even though the payload varies

Node	Cut	device	uplink	queue	server	residual
11 (A)	`k0`	19.4	7.6	0.1	4.3	1.1
11 (A)	`k_shallow`	**154.5**	9.7	0.1	1.5	1.1
11 (A)	`k_deep`	**180.6**	7.0	0.1	1.1	1.2
21 (B)	`k0`	22.3	8.5	0.1	4.1	1.1
21 (B)	`k_shallow`	**543.9**	11.0	0.1	1.6	1.2
21 (B)	`k_deep`	**708.1**	9.1	0.1	1.3	1.3
31 (C)	`k0`	24.3	11.9	0.1	4.2	1.1

Table 5: Mean stage breakdown in ms. The stages sum to the total.

5× between `k_deep` and `k_shallow`. At these sizes airtime is per-packet overhead rather than bytes, so the data-inflation effect of Table 3 is real and costs almost nothing in time.

4.5 The cost of running the fleet together

Policy	Node	Isolated (ms)	Concurrent (ms)	Difference
`k0`	11 (A)	32.6	42.4	**+30.2%**
`best` = `k0`	11 (A)	32.6	45.8	**+40.7%**
`k_shallow`	11 (A)	166.8	174.1	+4.4%
`k_deep`	11 (A)	190.0	193.5	+1.8%

Table 6: The same node running the same cut, measured alone versus with the whole fleet active. Policies that lean on the shared server tail pay an order of magnitude more for concurrency than policies that compute on the device.

The three nodes share one single-worker server tail, so a policy whose tail work is expensive makes every other node wait. The effect is there and in the predicted direction: splitting genuinely does buy immunity to server contention. It does not change the ranking, since `k0` at 42.4 ms with its penalty still beats `k_shallow` at 174.1 ms by 4.1×.

4.6 How much slower would the server have to be?

The server here is far faster than an MCU-class edge device, which favours offload. Holding the measured `device`, `uplink` and `residual` stages fixed and scaling the measured `server` stage gives the crossover where a split cut would overtake `k0`:

Node	Takes over	Server slower by	`k0` tail at crossover
11 (A)	`k_shallow`	51×	218 ms
21 (B)	`k_shallow`	217×	881 ms
31 (C)	none	no crossover	

Table 7: A projection over measured stages, not a measurement. Queueing is excluded, which makes every crossover here conservative.

For scale, a Raspberry Pi 4, the server this study was originally designed around, runs this model in roughly 10–25 ms. Splitting would still have lost there, by about an order of magnitude in required server slowness, so the result does not depend on the substitute server.

4.7 Reliability

Run	Requests	OK	TIMEOUT	CRC	LOST	Reboots
sweep	1,400	1,400	0	**0**	0	0
`k0`	603	600	0	**0**	3	0
`k_shallow`	610	600	9	**0**	1	0
`k_deep`	600	600	0	**0**	0	0
`best`	600	600	0	**0**	0	0

Table 8: Zero corrupted deliveries across 3,813 requests; no node rebooted during any run. Worst failure rate 1.6%.

Reaching this took three protocol fixes that only hardware exposed. The device blocked for exactly the task-watchdog timeout waiting for a result, so any request that never returned rebooted the node. A busy device silently dropped an assignment for a different request, so one lost result cost three unacked assignments. And the device's idle timeout outlived the orchestrator's own deadline, leaving it deaf after the server had moved on. All three were invisible in simulation, because in simulation nothing is ever lost.

5 Discussion

A single static policy is sufficient for this model on this fleet. The sweep's argmin is `k0` for all three nodes, and the `best` policy is by construction indistinguishable from `k0`. The orchestration machinery has nothing to decide.

Splitting trades server work for device work, and transfer for compute. On this fleet the exchange rate is ruinous: the head costs 121–668 ms on the MCU to save 2.7–3.0 ms on the server, over a link where the entire transfer takes 7–12 ms whichever cut produced it.

The heterogeneity is real and large. Tier B needs 4.2× tier A's time for identical work, and tier C cannot split at all. But the correct policy is nevertheless identical on all three tiers, so the premise that motivates per-device split selection, that different devices want different cuts, does not hold here.

Generalising in one specific direction: split inference pays when device compute is cheap relative to the server, and MobileNetV2-0.35 on an ESP32 is the opposite regime. The head is 12 of 67 operations and costs 121 ms on the best board in the fleet; the whole model costs 3.45 ms on one 400 MHz x86 core. Any argument for splitting this workload must begin by finding a server two orders of magnitude slower, or a device two orders of magnitude faster. A much larger model whose server cost is seconds, a far slower link, or a device with hardware NN acceleration would each change that arithmetic, but none is a small extrapolation from what was measured here.

6 Threats to validity

The server is an x86 laptop, not an MCU-class edge box. The Raspberry Pi the design called for was unavailable. Capping the CPU to 400 MHz brings the tail to 3.45 ms for the whole model, from 0.29 ms uncapped, with a tight distribution, but it is still far faster than a Pi. The bias favours offload; Table 7 quantifies how far it would have to move to matter, and the answer is far enough that the conclusion survives.

Single session, no repeats. One run per policy. Run-to-run spread on repeated identical runs was 10–28% on the mean, so differences of a few percent should not be read as real. The effects reported here are 4–15×.

The model is a placeholder. An ImageNet-initialised MobileNetV2-0.35 with an untrained head. Architecture, quantization and tensor shapes are those of the real model, so all latency and byte measurements are valid; its accuracy is meaningless and no accuracy claim is made.

Stored images, not live capture. All nodes read the same 50 images from flash, which makes the comparison controlled but omits camera capture cost. The flash read is also not quite symmetric: `k0` reads a 1.8 KB JPEG and the split cuts a 27,648 B raw tensor, costing 17–37 ms against 32–41 ms, because the cost is dominated by file opening rather than throughput. That ∼10 ms asymmetry is reported in the `device` column and is far too small to change any ordering.

Loss injection was one-directional. Injected loss reached the control path but not the tensor fragments, so the selective-repeat path is verified in simulation at five loss levels rather than on the radio.

Three nodes are not radio contention. This study says nothing about what happens when many devices transmit simultaneously.

7 Conclusion

No split point beat full offload on any board, cutting deeper made things worse, and the weakest board could not split at all. On-device inference dominates the budget by two orders of magnitude while transfer is effectively constant across cuts, so the parameter this project set out to tune has almost no leverage. The negative result is quantified rather than asserted: the server would need to be 51× slower before the shallow cut broke even, roughly an order of magnitude beyond the Pi-class server the study was designed around.